UNIVERSIDADE FEDERAL DE SERGIPE

Árvore geradora mínima

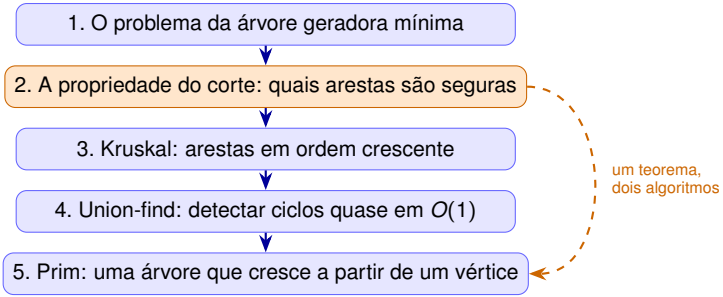
Os algoritmos de Kruskal e de Prim

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

3 de novembro de 2026

O que você vai aprender hoje



Na aula passada, o argumento de troca provou Huffman. Hoje ele prova, de uma vez, dois algoritmos gulosos em grafos.

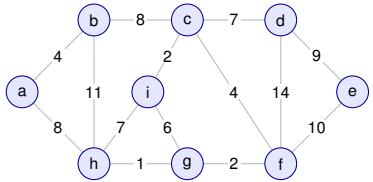
Para aprofundar: Cormen et al. (2009), cap. 23 [2]; Kleinberg e Tardos (2006), §4.5 [7].

O problema da árvore geradora mínima

Por que precisamos de uma árvore geradora mínima?

Problema: ligar nove bairros por fibra óptica. Cada trecho possível tem um custo de instalação. Que trechos instalar para que *todos* os bairros se comuniquem gastando o *mínimo*?

Não vale a pena instalar um trecho que fecha um ciclo: dá para tirá-lo sem desconectar ninguém e o custo cai. A resposta é uma **árvore**.



O exemplo condutor da aula: 9 vértices, 14 arestas com pesos. Foi assim que o problema nasceu: Borůvka (1926) planejava a rede elétrica da Morávia.

Para aprofundar: Cormen et al. (2009), §23.1, Figura 23.1 [2]; Graham e Hell (1985) [6].

Árvore geradora mínima: definição

Definição

Seja $G = (V, E)$ um grafo não dirigido **conexo** com pesos $w(u, v)$ nas arestas. Uma **árvore geradora** de G é um $T \subseteq E$ sem ciclos que liga todos os vértices. Seu peso é $w(T) = \sum_{(u,v)\in T} w(u, v)$. Uma **árvore geradora mínima** (AGM, em inglês *MST*) é uma árvore geradora de peso mínimo.

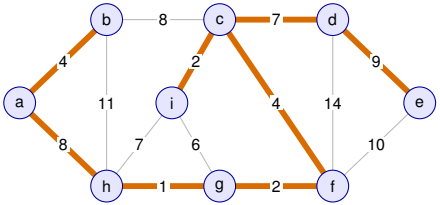
Intuição

"Geradora": alcança todos os vértices. "Árvore": nenhuma aresta sobrando. Toda árvore geradora tem exatamente $|V| - 1$ arestas; o que muda entre elas é *quais*.

Notação: $n = |V|$ e $m = |E|$. Se G for desconexo, pede-se uma *floresta* geradora mínima.

Para aprofundar: Cormen et al. (2009), §23.1 e Apêndice B.5 [2]; Kleinberg e Tardos (2006), §4.5 [7].

Exemplo: uma AGM do grafo condutor



Em laranja, $n - 1 = 8$ arestas:

$$1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37.$$

Nenhuma árvore geradora deste grafo pesa menos que 37.

Observe que a aresta (a, h), de peso 8, poderia ser trocada por (b, c), também de peso 8: o resultado é outra árvore geradora de peso 37. **A AGM não precisa ser única** — ela é única quando os pesos são todos distintos.

Para aprofundar: Cormen et al. (2009), §23.1, Figura 23.1 [2]; Kleinberg e Tardos (2006), §4.5 (pesos distintos) [7].

Por que não testar todas as árvores?

Fórmula de Cayley

O grafo completo K_n tem exatamente n^{n-2} árvores geradoras.

Mesmo num grafo esparso, a quantidade cresce exponencialmente. No exemplo condutor, há $\binom{14}{8} = 3003$ conjuntos de 8 arestas para testar; num grafo de rodovias com milhares de cidades, não há como.

Para aprofundar: Cayley (1889) [1]; Cormen et al. (2009), §23.1 [2].

n	árvores geradoras de K_n
4	16
9	4 782 969
20	$\approx 2{,}6 \times 10^{23}$
100	10^{196}

A boa notícia: aqui o guloso funciona, e roda em $O(m \log n)$.

Recapitulando: o problema

O que aprendemos

- Uma árvore geradora liga os n vértices com $n - 1$ arestas e nenhum ciclo.
- A AGM minimiza a soma dos pesos; no exemplo condutor, o mínimo é 37.
- Pode haver várias AGMs quando há pesos repetidos.
- Enumerar árvores é inviável: K_n tem n^{n-2} delas.

Para aprofundar: Cormen et al. (2009), §23.1 [2].

A propriedade do corte

Por que precisamos de um critério de segurança?

Problema: um algoritmo guloso vai juntar arestas uma a uma, sem voltar atrás. Como saber que uma aresta pode entrar *sem estragar* a chance de chegar a uma AGM?

É a pergunta da aula passada, a **escolha gulosa**, agora em grafos.

Para aprofundar: Cormen et al. (2009), §23.1, GENERIC-MST [2].

Solução: manter um conjunto A de arestas com o invariante

A está contido em alguma AGM

e só acrescentar arestas **seguras**: arestas (u, v) tais que $A \cup \{(u, v)\}$ ainda está contido em alguma AGM.

Quando $|A| = n - 1$, o próprio A é uma AGM.

Cortes e arestas leves: definição

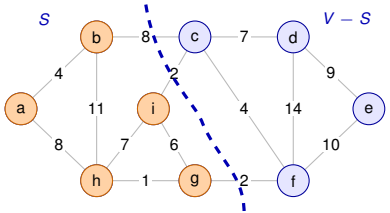
Definição

Um **corte** $(S, V - S)$ é uma partição dos vértices em dois lados. Uma aresta **cruza** o corte se tem uma ponta em cada lado. O corte **respeita** A se nenhuma aresta de A o cruza. Uma aresta é **leve** se tem o menor peso entre as que cruzam o corte.

Intuição

Qualquer árvore geradora precisa atravessar o corte pelo menos uma vez. A travessia mais barata é a candidata natural.

Para aprofundar: Cormen et al. (2009), §23.1, Figura 23.2 [2].



$S = \{a, b, h, i, g\}$. Cruzam o corte: (b, c): 8, (c, i): 2, (g, f): 2. Há duas arestas leves, ambas de peso 2.

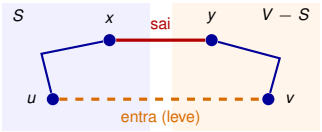
O teorema do corte

Teorema

Seja A contido em alguma AGM de G, seja $(S, V - S)$ um corte que respeita A e seja (u, v) uma aresta leve que cruza esse corte. Então (u, v) é **segura** para A.

Prova (troca). Seja T uma AGM com $A \subseteq T$. Se $(u, v) \in T$, acabou. Senão, o caminho de u a v em T cruza o corte em alguma aresta (x, y), que não está em A (o corte respeita A). Troque: $T' = T - \{(x, y)\} \cup \{(u, v)\}$.

T' é árvore geradora (o ciclo criado por (u, v) foi quebrado) e, como (u, v) é leve, $w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$. Logo T' é AGM e contém $A \cup \{(u, v)\}$. □



Em azul, o caminho de u a v em T.

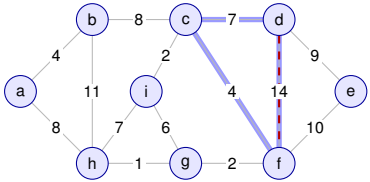
Para aprofundar: Cormen et al. (2009), §23.1, Teorema 23.1 e Figura 23.3 [2]; Kleinberg e Tardos (2006), §4.5, (4.17) [7].

O outro lado: a propriedade do ciclo

Propriedade do ciclo

Se e é a aresta de peso *estritamente maior* de algum ciclo C de G, então e não pertence a nenhuma AGM.

Prova (troca, de novo). Se uma AGM T contivesse e, tirar e partiria T em dois pedaços: um corte. O resto de C volta de um lado ao outro, então outra aresta e' de C cruza esse corte, e $T - \{e\} \cup \{e'\}$ é mais leve que T: absurdo. □



No ciclo c–d–f, a aresta (d, f): 14 é a mais pesada: nenhuma AGM a usa.

Corte: certifica que uma aresta **entra**. Ciclo: certifica que uma aresta **fica de fora**.

Para aprofundar: Kleinberg e Tardos (2006), §4.5, (4.20) [7]; Dasgupta et al. (2008), §5.1 [4].

O algoritmo genérico

AGM-GENÉRICO(G, w)

$A \leftarrow \emptyset$
enquanto A não for árvore geradora:
  escolha um corte que respeita A
  $(u, v) \leftarrow$ uma aresta leve do corte
  $A \leftarrow A \cup \{(u, v)\}$
devolva A

Pelo teorema do corte, o invariante vale em todas as iterações.

O teorema deixa uma liberdade: **qual corte** usar. Cada escolha dá um algoritmo.

	corte usado
Kruskal	o componente de A que contém u, contra o resto
Prim	os vértices da árvore que está crescendo, contra o resto

A prova dos dois é a mesma; muda a estrutura de dados que acha a aresta leve depressa.

Para aprofundar: Cormen et al. (2009), §23.1, GENERIC-MST e Corolário 23.2 [2].

Recapitulando: a propriedade do corte

O que aprendemos

- Uma aresta é segura se não impede chegar a uma AGM.
- **Corte:** a aresta leve de um corte que respeita A é segura.
- **Ciclo:** a aresta mais pesada de um ciclo nunca entra.
- As duas provas são argumentos de troca, como em Huffman.
- Kruskal e Prim são o mesmo algoritmo genérico com cortes diferentes.

Para aprofundar: Cormen et al. (2009), §23.1 [2]; Kleinberg e Tardos (2006), §4.5 [7].

O algoritmo de Kruskal

Por que começar pelas arestas mais leves?

Problema: queremos as arestas baratas, mas uma aresta barata pode fechar um ciclo com as que já foram escolhidas.

Pela propriedade do ciclo, quando uma aresta fecha ciclo com arestas *mais leves* já escolhidas, ela é dispensável.

Solução (Kruskal, 1956):

1. ordene as arestas por peso crescente;
2. percorra a lista; aceite a aresta se ela liga **dois componentes diferentes** da floresta A e rejeite se fecha ciclo;
3. pare quando $|A| = n - 1$.

A é uma **floresta** que vai fundindo componentes até virar uma árvore só.

Para aprofundar: Kruskal (1956) [8]; Cormen et al. (2009), §23.2, MST-KRUSKAL [2]; Levitin (2012), §9.2 [9].

Kruskal: por que está correto

Teorema

O algoritmo de Kruskal devolve uma árvore geradora mínima.

Aresta aceita (u, v)**.** Seja S o componente de A que contém u. O corte $(S, V - S)$ respeita A, e (u, v) o cruza. Qualquer aresta mais leve que cruzasse esse corte já teria sido examinada e aceita (ligava componentes diferentes). Logo (u, v) é leve: é segura pelo teorema do corte.

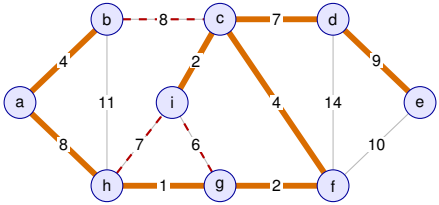
Aresta rejeitada (u, v)**.** Ela fecha um ciclo com arestas de A, todas examinadas antes, portanto de peso $\leq w(u, v)$. Ela não faz falta: A já liga u a v.

Fim. Como G é conexo, alguma aresta liga dois componentes enquanto houver mais de um. O laço termina com uma árvore só.

Para aprofundar: Cormen et al. (2009), §23.2 e Corolário 23.2 [2]; Kleinberg e Tardos (2006), §4.5, (4.18) [7].

Exemplo: Kruskal no grafo condutor

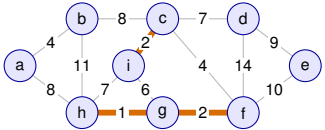
passo	aresta	peso	decisão
1	(g, h)	1	aceita
2	(c, i)	2	aceita
3	(f, g)	2	aceita
4	(a, b)	4	aceita
5	(c, f)	4	aceita
6	(g, i)	6	ciclo *g f c i*
7	(c, d)	7	aceita
8	(h, i)	7	ciclo
9	(a, h)	8	aceita
10	(b, c)	8	ciclo
11	(d, e)	9	aceita: $\|A\| = 8$



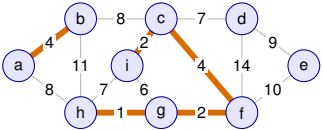
Em laranja, as aceitas; tracejadas em vermelho, as rejeitadas. As arestas (e, f), (b, h) e (d, f) nem são examinadas: o laço para no passo 11. Peso 37.

Para aprofundar: Cormen et al. (2009), §23.2, Figura 23.4 [2].

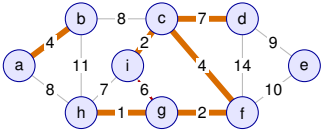
A floresta de Kruskal ao longo do tempo



depois do passo 3



depois do passo 5



depois do passo 7

No passo 3 há 6 componentes: $\{g, h, f\}$, $\{c, i\}$ e quatro vértices isolados. No passo 5, $\{c, f, g, h, i\}$ e $\{a, b\}$ se formaram. A aresta (g, i): 6 do passo 6 liga dois vértices *do mesmo componente*, por isso fecha um ciclo.

A pergunta que sobra: **como saber depressa** se u e v já estão no mesmo componente?

Para aprofundar: Cormen et al. (2009), §23.2, Figura 23.4 [2].

Recapitulando: Kruskal

O que aprendemos

- Kruskal examina as arestas em ordem crescente de peso.
- Aceita se liga dois componentes; rejeita se fecha ciclo.
- A aresta aceita é leve no corte do seu componente: é segura.
- No exemplo, aceita 8 de 11 arestas examinadas e chega a 37.
- Falta uma estrutura que responda “mesmo componente?” rapidamente.

Para aprofundar: Cormen et al. (2009), §23.2 [2].

Union-find: conjuntos disjuntos

Por que precisamos de uma estrutura nova?

Problema: para cada uma das m arestas, decidir se u e v estão no mesmo componente de A.

Com uma busca em profundidade (DFS) em A a cada aresta, gastamos $O(n)$ por teste e $O(mn)$ no total. Mais lento que ordenar.

Solução: componentes só se fundem. Basta uma estrutura de **conjuntos disjuntos**:

- $\texttt{cria}(v)$: $\{v\}$ vira um conjunto;
- $\texttt{find}(v)$: o *representante* do conjunto de v;
- $\texttt{une}(u, v)$: funde os dois conjuntos.

Mesmo componente $\iff \texttt{find}(u) = \texttt{find}(v)$.

Para aprofundar: Cormen et al. (2009), §21.1 [2]; Kleinberg e Tardos (2006), §4.6 [7].

Floresta de conjuntos disjuntos: definição

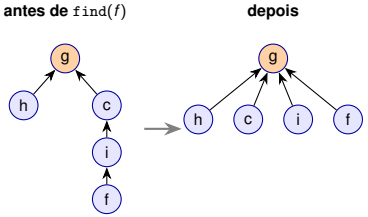
Definição

Cada conjunto é uma árvore enraizada guardada num vetor `pai[]`; a raiz é o representante e tem `pai[r] == r`. `find` sobe até a raiz; `une` pendura uma raiz na outra.

Duas heurísticas

União por tamanho: pendure a árvore *menor* na maior; a altura fica $\leq \log_2 n$.

Compressão de caminho: na volta do `find`, ligue todos os vértices do caminho direto à raiz.



O próximo `find` de *f*, *i* ou *c* sobe um passo só.

Para aprofundar: Cormen et al. (2009), §21.3, Figura 21.5 [2]; Levitin (2012), §9.2 [9].

Union-find e Kruskal em C

```c
int pai[MAXN], tam[MAXN];

int find( int x) {           /* com compressao */
   if (pai[x] != x)
      pai[x] = find( pai[x]);
   return pai[x];
}

void une( int a, int b) {    /* a, b: raizes */
   if (tam[a] < tam[b]) {    /* uniao por tamanho */
      int t = a; a = b; b = t;
   }
   pai[b] = a;
   tam[a] += tam[b];
}
```

```c
typedef struct { int u, v, w; } Aresta;

/* grava em T[] as arestas da AGM; devolve quantas */
int kruskal( int n, int m, Aresta e[], Aresta T[]) {
   qsort( e, m, sizeof (Aresta), cmp_peso);
   for (int v = 0; v < n; ++v) {
      pai[v] = v;  tam[v] = 1;
   }
   int k = 0;
   for (int i = 0; i < m && k < n-1; ++i) {
      int a = find( e[i].u), b = find( e[i].v);
      if (a != b) {          /* componentes distintos */
         T[k++] = e[i];
         une( a, b);
      }
   }
   return k;
}
```

Para aprofundar: Cormen et al. (2009), §21.3 e §23.2, MST-KRUSKAL [2]; Feofiloff (2020), Kruskal [5].

Quanto custa Kruskal?

etapa	custo
ordenar as m arestas	$O(m \log m)$
$2m$ `find` e $n-1$ `une`	$O(m\,\alpha(n))$
total	$O(m \log m) = O(m \log n)$

Como $m \leq n^2$, vale $\log m \leq 2 \log n$. A ordenação domina.

$\alpha(n)$ é a inversa da função de Ackermann: cresce tão devagar que $\alpha(n) \leq 4$ para qualquer n que caiba no universo observável. Na prática, cada operação custa $O(1)$.

heurísticas	m operações
nenhuma	$O(mn)$ no pior caso
só união por tamanho	$O(m \log n)$
as duas	$O(m\,\alpha(n))$

Para aprofundar: Cormen et al. (2009), §21.4 e §23.2 [2]; Tarjan (1975) [12].

Recapitulando: union-find

O que aprendemos

- Componentes que só se fundem cabem numa floresta de conjuntos disjuntos.
- `find` sobe até a raiz; `une` pendura uma raiz na outra.
- União por tamanho e compressão de caminho levam a $O(\alpha(n))$ amortizado.
- Com elas, Kruskal custa $O(m \log n)$, dominado pela ordenação.

Para aprofundar: Cormen et al. (2009), cap. 21 [2]; Kleinberg e Tardos (2006), §4.6 [7].

O algoritmo de Prim

Por que crescer uma árvore só?

Problema: Kruskal precisa ordenar todas as arestas e manter uma floresta inteira. Em grafos densos, $m \approx n^2$ arestas.

Outra maneira de escolher o corte: o lado S é *uma* árvore que começa num vértice r e só cresce.

Solução (Jarník, 1930; Prim, 1957):

1. $S \leftarrow \{r\}$;
2. repita $n - 1$ vezes: escolha a aresta **mais leve** com uma ponta em S e outra fora, e traga a ponta de fora para S.

Cada aresta escolhida é leve no corte $(S, V - S)$, que respeita a árvore atual: é segura pelo teorema do corte.

Para aprofundar: Prim (1957) [10]; Cormen et al. (2009), §23.2, MST-PRIM [2]; Levitin (2012), §9.1 [9].

Prim: a chave de cada vértice

Definição

Para cada vértice v fora da árvore, $chave[v]$ é o menor peso de uma aresta que liga v à árvore (∞ se não houver), e $pai[v]$ é a outra ponta dessa aresta.

Intuição

A chave é o preço de trazer v para a árvore agora. Em cada passo entra o vértice mais barato u; depois, só as chaves dos *vizinhos* de u podem baixar:

$$chave[v] \leftarrow \min\big(chave[v],\ w(u,v)\big).$$

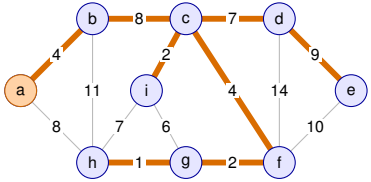
As chaves formam uma fila de prioridade, como em Huffman. Guarde a regra: Dijkstra, na próxima aula, troca $w(u,v)$ por $dist[u] + w(u,v)$.

Para aprofundar: Cormen et al. (2009), §23.2, MST-PRIM [2]; Dasgupta et al. (2008), §5.1 [4].

Exemplo: Prim a partir de *a*

entra	*b*	*c*	*d*	*e*	*f*	*g*	*h*	*i*
a	**4**	∞	∞	∞	∞	∞	**8**	∞
b (4)	–	**8**	∞	∞	∞	∞	8	∞
c (8)	–	–	**7**	∞	**4**	∞	8	**2**
i (2)	–	–	7	∞	4	**6**	**7**	–
f (4)	–	–	7	**10**	–	**2**	7	–
g (2)	–	–	7	10	–	–	**1**	–
h (1)	–	–	7	10	–	–	–	–
d (7)	–	–	–	**9**	–	–	–	–
e (9)	–	–	–	–	–	–	–	–

Chaves depois de cada entrada; em negrito, as que baixaram.



Na 2.ª linha, *c* e *h* empatam com chave 8; entrou *c* (o de menor rótulo). A árvore usa (b, c) no lugar de (a, h): outra AGM, também de peso $4 + 8 + 2 + 4 + 2 + 1 + 7 + 9 = 37$.

Para aprofundar: Cormen et al. (2009), §23.2, Figura 23.5 [2].

Prim em C com matriz de adjacência

```
/* W[u][v] = peso, ou INF se nao ha aresta.
   Preenche pai[]: a AGM e {(v, pai[v]) : v != 0}. */
void prim( int n, int W[][MAXN], int pai[]) {
   int chave[MAXN];  bool dentro[MAXN];
   for (int v = 0; v < n; ++v) {
      chave[v] = INF;  pai[v] = -1;  dentro[v] = false;
   }
   chave[0] = 0;                          /* raiz r = 0 */
   for (int it = 0; it < n; ++it) {
      int u = -1;                         /* o mais barato fora */
      for (int v = 0; v < n; ++v)
         if (!dentro[v] && (u < 0 || chave[v] < chave[u]))
            u = v;
      dentro[u] = true;
      for (int v = 0; v < n; ++v)         /* vizinhos de u */
         if (!dentro[v] && W[u][v] < chave[v]) {
            chave[v] = W[u][v];  pai[v] = u;
         }
   }
}
```

Para aprofundar: Feofiloff (2020), Prim [5]; Levitin (2012), §9.1 [9].

Custo. São n iterações, e cada uma varre o vetor duas vezes: $\Theta(n^2)$, independente de m.

Para grafos **densos** ($m \approx n^2$) isso é ótimo: só ler a matriz já custa $\Theta(n^2)$.

Para grafos **esparsos**, troque a varredura por um heap mínimo de vértices indexado pela chave (exercício do tutorial).

Quanto custa Prim?

fila de prioridade	operações dominantes	total
vetor (varredura)	n extrações de $\Theta(n)$	$\Theta(n^2)$
heap binário	n extrações e até m reduções de chave, $O(\log n)$ cada	$O(m \log n)$
heap de Fibonacci	redução de chave em $O(1)$ amortizado	$O(m + n \log n)$

Grafo esparso ($m = O(n)$): heap binário dá $O(n \log n)$, e o vetor perde com $\Theta(n^2)$.

Grafo denso ($m = \Theta(n^2)$): o vetor dá $\Theta(n^2)$, e o heap binário perde com $O(n^2 \log n)$.

Sem heap com redução de chave, dá para inserir de novo o vértice com a chave menor e ignorar as cópias velhas ao extrair ("Prim preguiçoso"): até m itens no heap, ainda $O(m \log n)$.

Para aprofundar: Cormen et al. (2009), §23.2 [2]; Sedgewick e Wayne (2011), §4.3 (versões preguiçosa e ansiosa) [11].

Recapitulando: Prim

O que aprendemos

- Prim cresce uma única árvore a partir de uma raiz.
- A cada passo entra o vértice de menor chave, a aresta leve do corte.
- Com vetor custa $\Theta(n^2)$; com heap binário, $O(m \log n)$.
- No exemplo, chega a outra AGM de peso 37, com (b, c) no lugar de (a, h).

Para aprofundar: Cormen et al. (2009), §23.2 [2].

Fechando

Kruskal ou Prim?

	Kruskal	Prim
corte usado	componente de u	árvore atual
estrutura auxiliar	ordenação + union-find	fila de prioridade de vértices
estado intermediário	floresta	uma árvore
tempo	$O(m \log n)$	$\Theta(n^2)$ (vetor) ou $O(m \log n)$ (heap)
melhor quando	esparso, arestas já em lista	denso, matriz de adjacência

Agrupamento. Pare Kruskal quando restarem k componentes: eles formam o agrupamento em k grupos de *espaçamento máximo* (a menor distância entre grupos é a maior possível).

História. Borůvka (1926) já resolvia o problema com outro guloso, que funde todos os componentes em paralelo; Jarník (1930) descreveu o que hoje chamamos de Prim.

Para aprofundar: Kleinberg e Tardos (2006), §4.7 (agrupamento) [7]; Graham e Hell (1985) [6].

O guloso até aqui

problema	escolha gulosa	troca usada na prova	tempo
intervalos sem peso	termina mais cedo	1.º intervalo de O por a_1	$O(n \log n)$
códigos de prefixo	junta os dois menores	folhas x, y pelas mais fundas	$O(n \log n)$
AGM (Kruskal)	aresta mais leve sem ciclo	aresta (x, y) do caminho	$O(m \log n)$
AGM (Prim)	vértice de menor chave	aresta (x, y) do caminho	$O(m \log n)$

Nos quatro casos, a prova toma uma solução ótima qualquer e a aproxima da gulosa por trocas que não pioram. A estrutura de dados (ordenação, heap, union-find) é o que torna a escolha gulosa barata.

Próxima aula: caminhos mínimos com Dijkstra, que é o código de Prim com outra chave.

Para aprofundar: Cormen et al. (2009), cap. 16 e 23 [2]; Kleinberg e Tardos (2006), cap. 4 [7].

Referências I



Arthur Cayley.
A theorem on trees.
Quarterly Journal of Pure and Applied Mathematics, 23:376–378, 1889.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Paulo Feofiloff.
Algoritmos para grafos (em linguagem C).
Material didático online, DCC/IME-USP, 2020.
Capítulos sobre árvores geradoras de custo mínimo e os algoritmos de Prim e de Kruskal. Acesso em 30 out. 2026.

Referências II



Ronald L. Graham and Pavol Hell.

On the history of the minimum spanning tree problem.
Annals of the History of Computing, 7(1):43–57, 1985.



Jon Kleinberg and Éva Tardos.

Algorithm Design.
Pearson/Addison-Wesley, 2006.
Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Joseph B. Kruskal.

On the shortest spanning subtree of a graph and the traveling salesman problem.
Proceedings of the American Mathematical Society, 7(1):48–50, 1956.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Robert C. Prim.

Shortest connection networks and some generalizations.
Bell System Technical Journal, 36(6):1389–1401, 1957.

Referências III



Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley, 4 edition, 2011.



Robert Endre Tarjan.
Efficiency of a good but not linear set union algorithm.
Journal of the ACM, 22(2):215–225, 1975.